



Universidad Veracruzana

INGENIERÍA BIOMÉDICA

INTERFAZ PARA UN FILTRO DE FRECUENCIA CARDIÁCA.

Proyecto.

Autores:

Caballero Trinidad Flor Isabel.

Cortés Ramírez Samuel Jefte.

García García Ian Pablo.

Pérez Roldan Eduardo Alejandro.

Cabañas Alba Alejandro.

Alfonso Gamboa Rubén.

Capítulo 1

Parcial 2

1.1. keywords

Frecuencia cardiaca, Interfaz, Filtro de frecuencia cardiaca, Interfaz para un filtro, SPython, Interfaz Médica, Ingeniería Biomédica, Arduino.

1.2. Introducción.

El siguiente proyecto tiene como finalidad responder a la pregunta ¿Cómo desarrollar una interfaz que facilite la visualización y procesamiento de datos de la frecuencia cardiaca?, teniendo como principal objetivo visualizar la frecuencia cardiaca del usuario mediante una interfaz a partir del lenguaje de programación Python [5].

Los temas que se abordaran y se trabajaran para la realización óptima del proyecto son: desarrollar una interfaz amigable con el usuario con la finalidad de interpretar los datos de manera precisa, garantizando la funcionalidad del sistema diseñando una interfaz para una gráfica; con el lenguaje de programación Python. Se busca leer y escribir ficheros, desarrollar un código legible para el programador y una forma de almacenar los datos recolectados, con el propósito de calcular la media aritmética para añadir funcionalidad que permita al usuario calcular su recuperación cardiaca. Este proyecto esta dirigido a un público general busca monitorear su frecuencia cardiaca.

Se llevará a cabo el proyecto con la finalidad de adquirir conocimiento sobre la creación de una interfaz en el lenguaje de programación Python, con la ayuda de una practica anterior relacionada a la Ingeniería Biomédica, se obtendrán lecturas de medición para la realización de prueba predictiva en índices para medir la salud cardiaca, se diseñara una versión base de una interfaz para el almacenamiento de señales biomédicas, con la finalidad de utilizarlas en un futuro y poder manipularlas, también se trabajara en conjunto con compañeros de la carrera de Instrumentación Electrónica, para así reforzar el trabajo en equipo y crear un frontend y un backend.

Para filtrar la búsqueda de la información necesaria para poder realizar el proyecto, se procedió a una búsqueda sistematizada de evidencia siguiendo los criterios de inclusión: artículos, libros, tesis, publicadas desde el año 1990 hasta la actualidad, información relacionada a como crear una interfaz en el lenguaje de programación Python, información del funcionamiento y características de la frecuencia cardiaca.

En esta sección se elabora una revisión bibliográfica de los conceptos generales a partir de los cuales se sustenta este proyecto, los conceptos a considerar son: señales biomédicas, interfaz, base de datos, frontend, backend, lenguaje de programación Python, frecuencia cardiaca.

Señales biomédicas. ¿Qué es una señal biomédica? "Las señales biomédicas representan variables fisiológicamente relevantes de las que nos interesa su curso temporal. La variable puede ser un voltaje muy pequeño, como el electroencefalograma, o algunos órdenes de magnitud mayor, como el electrocardiograma, o puede ser una variable que originalmente no es eléctrica, como la presión o la temperatura" [8].

Interfaz. ¿Qué es una interfaz? la RAE la define como: "Conexión o frontera común entre dos aparatos o sistemas independientes, que permite el intercambio de información o la coordinación de sus funciones." [7]

Frontend y backend. ¿Qué es front-end y back-end?. El front-end es la parte que va a interactuar con el usuario, en el caso de nuestro proyecto será la interfaz, el back-end es la parte que contiene la infraestructura que le va a dar funcionalidad a el proyecto. [1]

Lenguaje de programación Python. Python es un lenguaje de programación de alto nivel, el cual ha agarrado bastante popularidad en nuestros tiempos gracias a su legibilidad, razón por la cual también es el más usado en cuanto a aprender a programar se refiere; razón por la cual hoy en día contamos con un sin fin de librerías y documentación al respecto. [5]

Frecuencia cardiaca. La frecuencia cardiaca o pulso, es la cantidad de veces que late por cierto periodo de tiempo, se puede medir en varias partes del cuerpo como la muñeca, cuello, pecho, entre otros. La frecuencia de un adulto suele encontrarse dentro de 60 – 100 latidos por minuto. [6]

1.3. Metodología.

Se utilizará el lenguaje de programación Python [5] para la aplicación principal. Se utilizará la librería "TkInter" [2] y la librería "Custom TkInter" [3] para el diseño de una interfaz amigable con el usuario.

Para el despliegue de las gráficas de pulso cardiaco se utilizará la librería matplotlib [4].

Para la obtención de las señales cardiacas se utilizará un sensor "Max 30100" (figura 1.2) y una placa arduino uno (figura 1.1) con el cual se medirá el pulso cardiaco de una persona; para la comunicación se utilizara un programa por aparte hecho en python. Posteriormente guardarlo en un archivo "csv", dicho archivo pasará por un filtro realizado por nuestros compañeros de instrumentación electrónica para así entregarnos una señal legible y más limpia; para posteriormente ser gráficada en nuestra aplicación.

Para una mejor organización y para un control de versiones, se usará git y github. Publicado en: <https://github.com/Benchito2003/Proyecto-programacion.git>.

Figura 1.1: Placa arduino uno

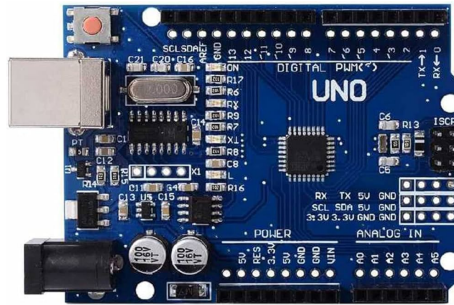


Figura 1.2: Sensor max 30100



1.3.1. Funcionamiento del código

El código se empezó a programar con ayuda de la programación orientada a objetos, donde cada ventana, frame o widget es en realidad una clase (o molde) hecho por nosotros, de manera que simplemente los mandamos a llamar, algunas características generales de estas clases son: que comparten la misma paleta de colores, comparten la misma fuente, sus módulos pueden ser ejecutados ahí mismo para testear que dicho módulo esté funcionando correctamente.

Para esto, la documentación de Custom Tkinter [3], cuenta con varios ejemplos ilustrativos de como podemos trabajar con sus clases orientadas a objetos, por

lo que nosotros lo que lo unico que se realizó fue heredar de éstas clases sus métodos y características (osease heredados).

colores

Figura: 1.3. Primeramente se hicieron varios conjuntos de colores en forma de listas, para poder escoger a futuro, el color que le quedaría mejor a nuestra interfaz, pudiendo establecer desde un principio la paleta con la que iremos trabajando.

config

Importamos custom tkinter [3]
El módulo config (figura 1.4) es un módulo donde creamos clases o moldes para nuestros botones, etiquetas, entre otros; dejando que hereden lo importante de custom tkinter [3] y después asignándoles nuestros propios "Valores por defecto". Les asignamos colores acorde a la paleta de colores asignada y ya están listos para ser usados por el resto del programa. Así se ahorra el repetir bastante código, y en caso de ser necesario un cambio hacia todas las etiquetas, se puede hacer directamente desde este módulo.

Frames

Para la construcción de cada frame (figura 1.5), solo se llama desde el modulo de config a clase, siguiendo el ejemplo de la documentación de custom tkinter [3]

funciones

En un móulo aparte llamado "funciones botones.py" (figura 1.6) , se dejaron las funciones más utilizadas por las demás frames a la hora de usar botones, como lo son: mostrar el siguiente frame, cerrar la ventana, etc.

ventanas emergentes

Para la creación de una ventana emergente (figura 1.7), se siguen los consejos de la documentación de custom tkinter [3], donde ya únicamente le agregamos funciones propias de cada ventana y le creamos frames a partir de las clases que previamente hicimos.

Módulo principal

El módulo principal es el cual se va a ejecutar y llamara como tal a todo el programa, ya listo para usar, en este caso se llama inicio.py" (figura 1.8)

1.3.2. resultados (previos)

Al ejecutar el módulo principal "inicio.py" (figura 1.8), se mostrara una imagen con la presentacion del programa (figura 1.9), posteriormente tenemos un menú (figura 1.10) en el cual podemos escoger si crear un nuevo usuario o entrar con uno ya creado a partir de su código de usuario. En caso de querer crear un usuario nuevo se abrirá una ventana para poder crear el usuario (figura 1.11). En caso de haber ingresado con un usuario, llegamos a una ventana principal 1.12 donde puede escoger entre hacer un nuevo registro, hacer una prueba de esfuerzo o graficar los registros que tenga.

Figura 1.3: módulo colores.py”

```
1  # Colección de colores
2  # Crear una arreglo para guardar los colores
3
4  ## Propuesta 1 de Rubén
5  white = "#FCFCFC"
6  mindaro = "#D2F898"
7  yellow = "#F6F930"
8  yet = "#2F2F2F"
9  black = "#000000"
10 paleta1 = [white, mindaro, yellow, yet, black]
11
12 ## Propuesta 2 de Ruben
13 rusian_violet = "#38023B"
14 tropical_indigo = "#A288E3"
15 columbia_blue = "#BBD5ED"
16 mint_green = "#CEFDFF"
17 tea_green = "#CCFFCB"
18 paleta2 = [rusian_violet, tropical_indigo, columbia_blue, mint_green, tea_green]
19
20 ## Propuesta 3 de Ruben
21 p3_c1 = "#090f13"
22 p3_c2 = "#171f25"
23 p3_c3 = "#752e2b"
24 p3_c4 = "#c90a02"
25 p3_c5 = "#f2eab7"
26 paleta3 = [p3_c1, p3_c2, p3_c3, p3_c4, p3_c5]
27
28
29 ## Propuesta 4 de Ruben
30 p4_c1 = "#00a0b0"
31 p4_c2 = "#6a4a3c"
32 p4_c3 = "#cc333f"
33 p4_c4 = "#eb6841"
34 p4_c5 = "#edc951"
35 paleta4 = [p4_c1, p4_c2, p4_c3, p4_c4, p4_c5]
36
37 p5_c1 = "#101942"
38 p5_c2 = "#80043a"
39 p5_c3 = "#f60c49"
40 p5_c4 = "#f09580"
41 p5_c5 = "#ffffff"
42 paleta5 = [p5_c1, p5_c2, p5_c3, p5_c4, p5_c5]
43
44 p6_c1 = "#dcca8"
45 p6_c2 = "#c4b291"
46 p6_c3 = "#ab9b7b"
47 p6_c4 = "#938364"
48 p6_c5 = "#7a6b4d"
49
50 paleta6 = [
51     p6_c1,
52     p6_c2,
53     p6_c3,
54     p6_c4,
55     p6_c5
56 ]
57
58 p7_c1 = "#7e9b9a"
59 p7_c2 = "#6e8a89"
60 p7_c3 = "#5d7978"
61 p7_c4 = "#4d6867"
62 p7_c5 = "#3c5756"
63 paleta7 = [
64     p7_c1,
65     p7_c2,
66     p7_c3,
67     p7_c4,
68     p7_c5
69 ]
```

Figura 1.4: Módulo config.py”

```

1 # Módulo para guardar configuraciones globales:
2 # Si queremos cambiar el color de la aplicación se hace desde aquí y no desde cada ventana
3 import colores
4 import customtkinter
5 from tkinter import font
6
7 # Colores escogidos
8 paleta = colores.paleta4
9 c1 = paleta[0] # Para el fondo
10 c2 = paleta[1] # Iconos 1
11 c3 = paleta[2] # Iconos 2
12 c4 = paleta[3] # Para los marcos
13 c5 = paleta[4] # Para las letras (contraste del fondo)
14 c_negra = "#000000" # Colores constantes
15 c_blanco = "#FFFFFF" # Colores constantes
16
17 # Fuente para todas las ventanas a menos que quieras una fuente específica
18 fuente = "Times New Roman"
19 t_fuente = 14 # tamaño de la fuente
20 # Fuente específica
21 # Fuente específica
22 # Alternativas a fuente
23 """ fuente_normal = font.Font(family="sans serif", size=14)
24 fuente_negra = font.Font(family="sans serif", size=14, weight="bold") """
25
26 # Estilo general de etiquetas
27 class CustomLabel(customtkinter.CTkLabel):
28     def __init__(self, master, **kwargs):
29         super().__init__(master, font=(fuente, t_fuente), bg_color=c1, fg_color=c1, text_color=c4, **kwargs)
30
31 # Estilo general de botones
32 class CustomButton(customtkinter.CTkButton):
33     def __init__(self, master, **kwargs):
34         super().__init__(master, font=(fuente, t_fuente), bg_color=c1, fg_color=c4, text_color=c1, border_color=c4, **kwargs)
35
36 # Estilo general de las entradas
37 self.e_usuario = CTkEntry(self, master, font=(fuente, t_fuente), bg_color=c1, fg_color=c1, text_color=c4, placeholder_text="Ingresar usuario", border_color=c4)
38
39 # Estilo general de las combobox
40 class CustomComboBox(customtkinter.CTkComboBox):
41     def __init__(self, master, valores, comando, **kwargs):
42         super().__init__(master, font=(fuente, t_fuente), bg_color=c1, fg_color=c1, text_color=c4, border_color=c4, **kwargs)
43
44 # Configuración de frames
45 class CustomFrame(customtkinter.CTkFrame):
46     def __init__(self, master, **kwargs):
47         super().__init__(master, fg_color=c1, bg_color=c1, **kwargs)
48
49 # Configuraciones del grid
50 self.columnconfigure([0, 1, 2], weight=1)
51 self.rowconfigure([0, 1, 2, 3, 4, 5], weight=1)

```

Figura 1.5: Ejemplo de creación de un frame

```

1 from config import *
2 import funciones_botones as f
3
4 class FrCaptura(CustomFrame):
5     def __init__(self, master, frame_siguiente, **kwargs):
6         super().__init__(master, **kwargs)
7
8         # Variables de control
9         self.tiempo=20
10
11        # Widgets
12        self.l_titulo = CustomLabel(self, text="Captura de frecuencia cardiaca")
13        self.l_titulo.grid(row=0, column=0, columnspan=3)
14
15        self.l_tiempo = CustomLabel(self, text=f"{self.tiempo}")
16        self.l_tiempo.configure(font=(fuente, 25), fg_color=c_blanco, text_color=c_negro, corner_radius=20)
17        self.l_tiempo.grid(row=1, column=1, sticky="nsew")
18
19        # Botones
20        self.b_iniciar = CustomButton(self, text="Iniciar", command= lambda: self.cuenta_regresiva(self.tiempo))
21        self.b_iniciar.grid(row=2, column=1)
22
23        self.b_siguiente = CustomButton(self, text="siguiente", command= lambda: f.mostrar_frame(frame_siguiente))
24
25        self.b_reiniciar = CustomButton(self, text="reiniciar prueba", command=self.reinicio)
26
27        self.b_salir = CustomButton(self, text="Salir", command= lambda: f.cerrar_ventana(master))
28        self.b_salir.grid(row=5, column=1, columnspan=2, sticky="e")
29
30        # Funciones
31        def cuenta_regresiva(self, tiempo):
32            """ Función para una cuenta regresiva :o """
33            self.b_iniciar.grid_forget()
34            if tiempo > 0:
35                self.l_tiempo.configure(text=f"{tiempo}")
36                self.after(1000, self.cuenta_regresiva, tiempo-1)
37                # La función se llama así misma cada 1000 ms y recibe de argumento el tiempo anterior menos 1
38            else:
39                self.l_tiempo.configure(text="Prueba terminada")
40                self.b_siguiente.grid(row=3, column=1)
41                self.b_reiniciar.grid(row=4, column=1)
42
43        def reinicio(self):
44            self.l_tiempo.configure(text=f"{self.tiempo}")
45            self.b_iniciar.grid(row=2, column=1)
46            self.b_siguiente.grid_forget()
47            self.b_reiniciar.grid_forget()
48
49
50 if __name__ == "__main__":
51     class App(CustomTkinter.CTk):
52         def __init__(self):
53             super().__init__()
54             self.geometry("700x500")
55             self.grid_rowconfigure(0, weight=1) # configure grid system
56             self.grid_columnconfigure(0, weight=1)
57
58             self.frame_prueba = CustomFrame(self)
59
60             self.my_frame = FrCaptura(self, self.frame_prueba)
61             self.my_frame.grid(row=0, column=0, padx=20, pady=20, sticky="nsew")
62
63
64     app = App()
65     app.mainloop()
66
67 """
68 Código obtenido de: https://customtkinter.tomschimansky.com/documentation/widgets/frame
69 """

```

Figura 1.6: Módulo "funciones botones.py"

```

1
2 # Para guardar las funciones de los botones y no repetir código
3 # Aclaración: No es necesario poner aquí todas las funciones, solo las que probablemente se vana a repetir en todas las ventanas, como la función de cerrar ventanas
4 import customtkinter
5
6 # Cerrar la ventana (puedes pasarle el atributo self para cerrar la ventana padre)
7 def cerrar_ventana(ventana_a_cerrar):
8     ventana_a_cerrar.destroy()
9
10 def abrir_ventana(ventana_padre, ventana_hija): # código de: https://customtkinter.tomschneider.com/documentation/windows/toplevelsf
11     if ventana_padre.ventana_abierta is None or not ventana_padre.ventana_abierta.winfo_exists(): # Si la ventana no existe
12         ventana_padre.ventana_abierta = ventana_hija(ventana_padre) # Crea una nueva ventana
13     else:
14         ventana_padre.ventana_abierta.focus()
15
16 def mostrar_frame(frame):
17     frame.place(relwidth=1, relheight=1)
18     frame.tkraise()
19
20 # Funciones para borrar/obtener datos de entrys
21 def obtener_texto(entry):
22     texto = entry.get()
23     return texto
24
25 def borrar_texto(entry):
26     entry.delete(0, 'end')
27
28 def reemplazar_texto(entry, texto):
29     entry.delete(0, 'end')
30     entry.insert(0, texto)
31
32
33 """ ## Funciones relacionadas al mouse (en proceso)
34 def cambiar_estilo(boton):
35     # función a cambiar cuando el mouse está encima
36     print("Hola mundo")
37
38
39 def restaurar_estilo(boton):
40     # Regresar al estilo que tenía antes de que el mouse estuviera ahí de encima
41     print("Adios mundo")
42 """
43
44 # Pruebas
45 if __name__ == "__main__":
46     import creacion_usuarios
47
48     class App_ventana_nueva(customtkinter.CTk):
49         def __init__(self, *args, **kwargs):
50             super().__init__(*args, **kwargs)
51             self.geometry("500x400")
52
53             self.button_1 = customtkinter.CTkButton(self, text="open toplevel", command= lambda: abrir_ventana(self, creacion_usuarios.V_nuevo_usuario))
54             self.button_1.pack(side="top", padx=20, pady=20)
55
56             self.ventana_abierta = None
57
58     app = App_ventana_nueva() # Para probar las ventanas nuevas
59     app.mainloop()
60

```

Figura 1.7: Ejemplo de creación de una ventana emergente

```
1 import customtkinter
2 import funciones_botones as f
3 from config import *
4 import nr_bienvenida, nr_capturar, nr_resultados
5
6 # Ventana para la creación de los registros
7 class VNuevoRegistro(customtkinter.CTkToplevel):
8     def __init__(self, *args, **kwargs):
9         super().__init__(*args, **kwargs)
10        ## Configuración inicial de la ventana
11        self.geometry("700x500")
12        self.minsize(700, 500)
13        self.title("Nuevo Registro")
14        self.config(bg=c1)
15        ## Configuraciones del grid
16        self.columnconfigure([0, 1, 2], weight=1)
17        self.rowconfigure([0,1,2,3,4,5], weight=1)
18
19        """
20        La ventana consta de 3 frames:
21        1. Bienvenida al usuario
22        2. Inicio de prueba
23        3. Resultado
24        """
25        self.frame3 = nr_resultados.FrResultados(self)
26        self.frame2 = nr_capturar.FrCaptura(self, self.frame3)
27        self.frame1 = nr_bienvenida.FrBienvenida(self, self.frame2)
28        self.frame1.grid(column=0, row=0, sticky="nsew", padx=50, pady=50)
29
30
31 # Clase de prueba
32 if __name__ == "__main__":
33     class App(customtkinter.CTk):
34         def __init__(self, *args, **kwargs):
35             super().__init__(*args, **kwargs)
36             self.geometry("500x400")
37
38             self.button_1 = customtkinter.CTkButton(self, text="open toplevel", command=self.open_toplevel)
39             self.button_1.pack(side="top", padx=20, pady=20)
40
41             self.toplevel_window = None
42
43         def open_toplevel(self):
44             if self.toplevel_window is None or not self.toplevel_window.winfo_exists():
45                 self.toplevel_window = VNuevoRegistro(self) # create window if its None or destroyed
46             else:
47                 self.toplevel_window.focus() # if window exists focus it
48
49
50     app = App()
51     app.mainloop()
52
53 """ Código de: https://customtkinter.tomschimansky.com/documentation/windows/toplevel """
```

Figura 1.8: Módulo inicio.py”

```

1  """
2  Creadores:
3  Caballero Trinidad Flor Isabel
4  Cabañas Alba Alejandro
5  Cortés Ramirez Samuel Joffe
6  Pérez Roldán Eduardo Alejandro
7  García García Ian Pablo
8  Alfonso Gamboa Rubén
9  """
10
11 from customtkinter import CTk
12
13 #Importar módulos propios:
14 import marco0
15 import creacion_usuarios
16 import login
17 import funciones_botones as f
18 import principal
19 from config import * # variables reservadas: c1, c2, c3, c4, c5, c_blanco, c_negro, fuente, t_fuente.
20
21
22 # Ventana principal
23 # Esta es la ventana que se ejecutará
24 class VInicio(CTk):
25     def __init__(self, *args, **kwargs):
26         super().__init__(*args, **kwargs)
27         # Propiedades de la ventana
28         self.geometry("500x200") # Geometría (Aun puede ser cambiada)
29         self.minsize(500, 200) # tamaño mínimo para que no pasen cosas extrañas ;)
30         self.config(bg = c1) # Un color de fondo
31         self.title("Life Rhythm")
32
33         # Configuración de filas y columnas para toda la ventana principal, permitiendolas centrar el contenido sin importar el tamaño de la ventana
34         self.columnconfigure(0, weight=1)
35         self.rowconfigure(0, weight=1)
36
37         # Ventanas
38         self.ventana_abierta = None # Variable que nos ayudará a saber si ya hay abierta una ventana nueva
39
40         # marcos
41         # marco 1: marco de la ventana principal
42         self.marco2 = principal.FrPrincipal(master=self)
43         # marco 1: ingreso de usuario
44         self.marco1 = login.FrLogin(self, self.marco2)
45         # Marco 0: Presentación del programa
46         self.marco0 = marco0.Presenteishon(master=self, marco_siguiente=self.marco1)
47         self.marco0.grid(column=0, row=0, sticky="nsew", padx=50, pady=50)
48
49 # Main
50
51
52 if __name__ == "__main__":
53     inicio = VInicio()
54     inicio.mainloop()

```

Figura 1.9: Presentación del programa

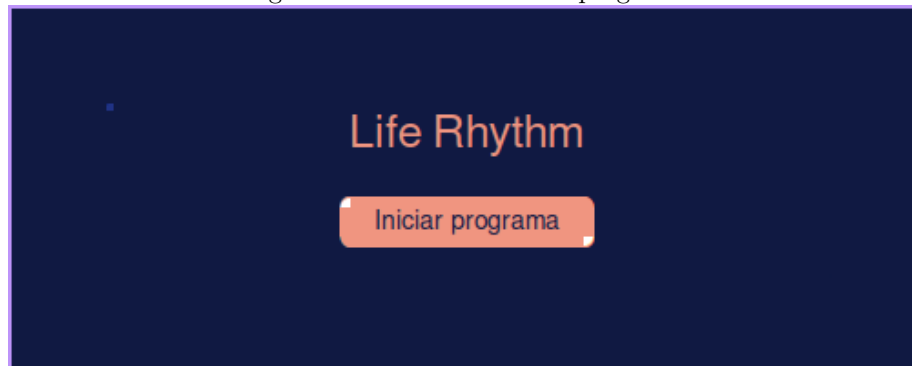
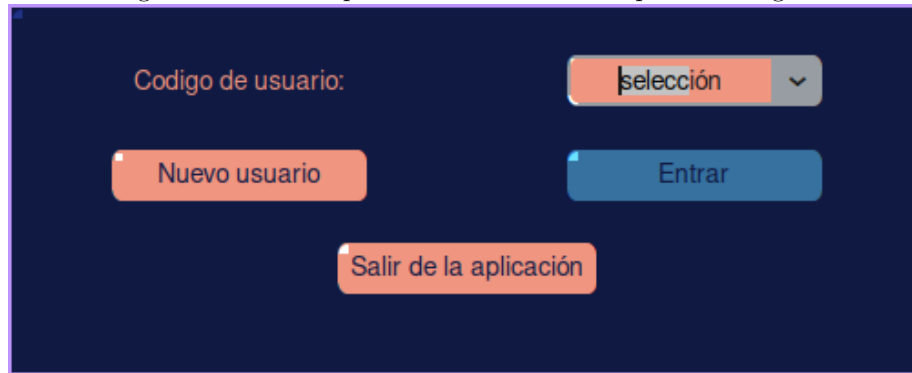


Figura 1.10: Menú que funciona como una especie de "login"



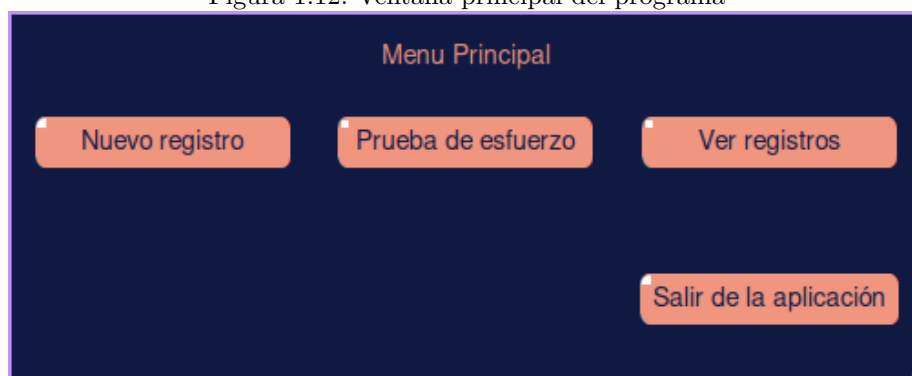
A login menu interface with a dark blue background. It features a label "Codigo de usuario:" followed by a dropdown menu showing "selección" with a downward arrow. Below this, there are three buttons: "Nuevo usuario" (orange), "Entrar" (blue), and "Salir de la aplicación" (orange).

Figura 1.11: Ventana para crear un nuevo usuario



A window for creating a new user with a dark blue background. It contains three input fields: "Nombre:" with the value "Panchito", "Apellido:" with the value "Perez", and "Edad:" with the value "Solo enteros". Below these fields are two buttons: "Generar usuario" (orange) and "cerrar" (orange).

Figura 1.12: Ventana principal del programa



Bibliografía

- [1] Frontend y backend: ¿qué son y cuáles son sus diferencias? — gluo.mx. <https://www.gluo.mx/blog/frontend-y-backend-que-son-y-cuales-son-sus-diferencias>. [Accessed 11-10-2024].
- [2] Interfaces gráficas de usuario con Tk — docs.python.org. <https://docs.python.org/es/3/library/tk.html>. [Accessed 15-10-2024].
- [3] Tariq Ali. customtkinter, 2021. URL <https://github.com/TomSchimansky/CustomTkinter>. Accedido: 2024-12-02.
- [4] John D. Hunter et al. Matplotlib: A 2d graphics environment, 2007. URL <https://matplotlib.org>. Accedido: 2024-12-02.
- [5] Python Software Foundation. Python programming language, 2023. URL <https://www.python.org>. Accedido: 2024-12-02.
- [6] fundaciondelcorazon. Frecuencia cardiaca — fundaciondelcorazon.com. <https://fundaciondelcorazon.com/prevencion/marcadores-de-riesgo/frecuencia-cardiaca.html>. [Accessed 16-10-2024].
- [7] RAE. interfaz — Diccionario de la lengua española — dle.rae.es. <https://dle.rae.es/interfaz>. [Accessed 11-10-2024].
- [8] Ennio A Vivaldi V and Pedro Maldonado A. Computadores en investigación superior biomédica: I. análisis de señales bioeléctricas. *Revista médica de Chile*, 129:955 – 962, 08 2001. ISSN 0034-9887. URL http://www.scielo.cl/scielo.php?script=sci_arttext&pid=S0034-98872001000800016&nrm=iso.